

SPECIFICA DEI TIPI DI DATO

- NumCart

stringa secondo standard

- CodSmart

stringa secondo standard azienda

- Targa

Stringa secondo standard

Italia ~ [A-Z]{2}[0-9]{3}[A-Z]{2}

- Denaro

(Valore: Reale ≥ 0 ,

Valuta: stringa secondo standard [EUR, USD, ecc...])

- Coordinate

(lat:-90..90, long:-180..180)

SPECIFICA DELLE CLASSI

- Classe Auto

ultima_manutenzione():Data

pre:

EXISTS manu |

auto_manu(this, manu)

post:

result soddisfa la seguente formula:

```

DATE_MANU = { d | EXISTS manu |
                                auto_manu(this, manu) AND
                                data(manu, d) }

AND

result = MAX( d | d in DATE_MANU )

```

- Classe Noleggio

[V.Noleggio.no_prima_rilascio_smartcard]

```

!EXISTS scard, rila, ini |
    inizio(this, ini) AND
    nol_smart(this, scard) AND
    rilascio(scard, rila) AND
    ini < rila

```

- Classe NoleggioTerminato

[V.NoleggioTerminato.no_prima_inizio]

```

!EXISTS ini, fin |
    inizio(this, ini) AND
    fine(this, fin) AND
    fin < ini

```

- Classe SocioDipendente

[V.SocioDipendente.scard_solo_se_ha_società]

```

!EXISTS societ, scard |
    smart_soc(scard, this) AND
    !soc_socio(societ, this)

```

[V.SocioDipendente.scard_solo_se_preaccettato]

```
!EXISTS scard |  
    smart_soc(scard, this) AND  
    !Pre-Accettato(this)
```

- Classe Sinistro

[V.Sinistro.dentro_intervallo_noleggio]

```
FORALL s, nol |  
    Sinistro(s) AND  
    nol_sini(nol, s) AND  
    NoleggioTerminato(nol) ->  
        EXISTS ist, ini, fin |  
            istante(s, ist) AND  
            inizio(nol, ini) AND  
            fine(nol, fin) AND  
            ini <= ist AND  
            ist <= fin
```

- Classe Socio

[V.Socio.no_scard_se_rifiutato]

```
!EXISTS scard |  
    smart_soc(scard, this) AND  
    SocioRifiutato(this)
```

abituale():Booleano

// Coming Soon

n_sin_no_controparte(d:Data):Inter0 >= 0

pre:

nessuna

post:

result soddisfa la seguente formula:

SIN = { s | EXISTS scard, nol, sini,
ist, di |
smart_soc(scard, this) AND
nol_smart(nol, scard) AND
nol_sini(nol, sini) AND
!SinistroControparte(sini) AND
istante(sini, ist) AND
data(ist, di) AND
oggi - d <= di }

AND
result = |SIN|

n_sin_con_controparte(d:Data):Inter0>=0

pre:

nessuna

post:

result soddisfa la seguente formula:

SIN = { s | EXISTS scard, nol, sini,
ist, di |
smart_soc(scard, this) AND
nol_smart(nol, scard) AND
nol_sini(nol, sini) AND

```

SinistroControparte(sini) AND
socio_responsabile(sini, true) AND
istante(sini, ist) AND
data(ist, di) AND
oggi - d <= di }

```

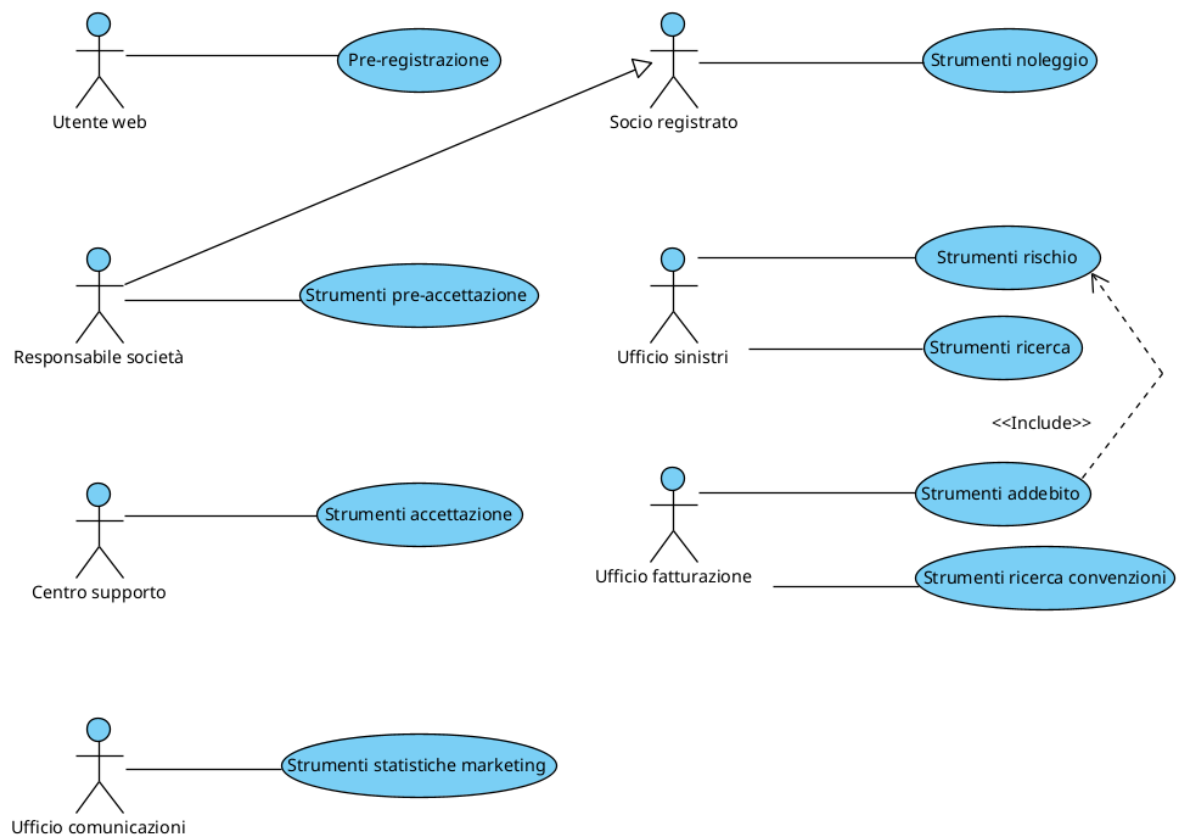
AND

```

result = |SIN|

```

DIAGRAMMA DEGLI USE-CASE



SPECIFICA DEGLI USE-CASE

- Use-Case Pre-registrazione

pre_registrazione_priv(

cf:CF,

nom:Stringa,

cog:Stringa,

nasc:Data,

res:Indirizzo,

codPat:CodPat,

scadPat:Data,

numCart:NumCart,

scadCart:Data

): SocioPrivato

pre:

!EXISTS s' |

cf(s', cf)

AND

!EXISTS s', pat' |

pat_soc(pat', s') AND

codice(pat', codPat)

AND

!EXISTS s', cart' |

cart_soc(cart', s') AND

numero(cart', numCart)

post:

il livello estensionale finale
differisce da quello iniziale
come segue:

nuovi elementi del dominio: soc, pat, cart
elementi rimossi dal dominio: nessuno

nuove ennuple:

- Socio(soc)
- cf(soc, cf)
- nome(soc, nom)
- cognome(soc, cog)
- nascita(soc, nasc)
- SocioPrivato(soc)
- residenza(soc, res)
- Patente(pat)
- codice(pat, codPat)
- scadenza(pat, scadPat)
- pat_soc(pat, soc)
- CartaDiCredito(cart)
- numero(cart, numCart)
- scadenza(cart, scadCart)
- cart_soc(cart, soc)

ennuple rimosse: nessuna

valore di ritorno: result = soc

pre_registrazione_dip(

cf:CF,

nom:Stringa

cog:Stringa

nasc:Data

codPat:CodPat

scadPat:Data

societ:Società

):SocioDipendente

pre:

!EXISTS s', pat', cod' |

Socio(s') AND

pat_soc(pat', s') AND

codice(pat', cod') AND

(cf(s', cf) OR codice(pat', codPat))

post:

il livello estensionale finale

differisce da quello iniziale

come segue:

nuovi elementi del dominio: soc, pat

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Socio(soc)

- cf(soc, cf)

- nome(soc, nom)

- cognome(soc, cog)

- nascita(soc, nasc)

- Patente(pat)
- codice(pat, codPat)
- scadenza(pat, scadPat)
- pat_soc(pat, soc)
- SocioDipendente(soc)
- soc_socio(societ, soc)

ennuple rimosse: nessuna

valore di ritorno: result = soc

- Use-Case Strumenti pre-accettazione

pre_accetta_dipendente(soc:SocioDipendente)

pre:

!SocioRifiutato(soc)

AND

!Pre-Accettato(soc)

post:

il livello estensionale finale

differisce da quello iniziale

come segue:

nuovi elementi del dominio: nessuno

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Pre-Accettato(soc)

ennuple rimosse: nessuna

valore di ritorno: nessuno

rifiuta_dipendente(soc:SocioDipendente)

pre:

!SocioRifiutato(soc)

AND

!Pre-Accettato(soc)

post:

il livello estensionale finale

differisce da quello iniziale

come segue:

nuovi elementi del dominio: nessuno

elementi rimossi dal dominio: nessuno

nuove ennuple:

- SocioRifiutato(soc)

- rifiuto(soc, adesso)

ennuple rimosse: nessuna

valore di ritorno: nessuno

- Use-Case Strumenti noleggio

inizia_noleggio(scard:SmartCard, aut:Auto, danni_not:Stringa[0..1]): Noleggio

pre:

sia s:Socio l'istanza dell'attore che
ha invocato lo use-case

smart_soc(scard, s)

AND

!EXISTS nol' |

nol_smart(nol', scard) AND

```
( !NoleggioTerminato(nol') OR  
(!NoleggioTerminato(nol') AND auto_nol(aut, nol')) )
```

post:

il livello estensionale finale
differisce da quello iniziale
come segue:

nuovi elementi del dominio: alpha
elementi rimossi dal dominio: nessuno
nuove ennuple:

- Noleggio(alpha)
- inizio(alpha, adesso)
- danni_not != null -> danni_notati(alpha, danni_not)
 - nol_smart(alpha, scard)
 - auto_nol(aut, alpha)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

termina_noleggio(nol:Noleggio):NoleggioTerminato

pre:

sia s:Socio l'istanza dell'attore
che ha invocato lo use-case

!NoleggioTerminato(nol)

AND

EXISTS scard |

smart_soc(scard, s) AND

nol_smart(nol, scard)

post:

il livello estensionale finale
differisce da quello iniziale

come segue:

nuovi elementi del dominio: nessuno
elementi rimossi dal dominio: nessuno
nuove ennuple:
 - NoleggioTerminato(nol)
 - fine(nol, adesso)
ennuple rimosse: nessuna
valore di ritorno: result = nol

- Use-Case Strumenti accettazione

accetta_socio(soc:Socio):SmartCard

pre:

!EXISTS scard' |
 smart_soc(scard', soc)
AND
SocioDipendente(soc) -> Pre-Acettato(soc)

post:

il livello estensionale finale
differisce da quello iniziale
come segue:

nuovi elementi del dominio: alpha
elementi rimossi dal dominio: nessuno
nuove ennuple:
 - SmartCard(alpha)
 - rilascio(alpha, adesso)
Un sistema esterno genererà
cod:CodSmart

- codice(alpha, cod)
- smart_soc(alpha, soc)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

rifiuta_socio(soc:Socio):

pre:

!SocioRifiutato(soc)

post:

il livello estensionale finale
differisce da quello iniziale
come segue:

nuovi elementi del dominio: nessuno
elementi rimossi dal dominio: nessuno
nuove ennuple:

- SocioRifiutato(soc)
- rifiuto(soc, adesso)

ennuple rimosse: nessuna

valore di ritorno: result = soc

- Use-Case Strumenti rischio

classe_rischio(soc:Socio): 0..10

pre:

nessuna

post:

questo use-case non modifica il
livello estensionale

result soddisfa la seguente formula:

```

    EXISTS nSin, nSinNoCon |
    n_sin_no_controparte(soc, oggi - "3 anni", nSin)
    AND
    n_sin_con_controparte(soc, oggi - "3 anni", nSinNoCon)
    AND
    tot = nSin + nSinNoCon
    AND
    tot = 0 -> result = 0
    AND
    (tot > 0 AND tot < 10) -> result = tot
    AND
    tot >= 10 -> result = 10

```

cerca_responsabile(aut:Auto, ist:DataOra):Socio[0..1]

pre:

nessuna

post:

questo use-case non modifica il

livello estensionale

result soddisfa la seguente formula:

```

result = { s | EXISTS scard, nol,
                                ini, fin |
                                smart_soc(scard, s) AND
                                nol_smart(nol, scard) AND
                                NoleggioTerminato(nol) AND
                                auto_nol(aut, nol) AND
                                inizio(nol, ini) AND
                                fine(nol, fin) AND
                                ist >= ini AND

```

```
ist <= fin }
```

Use-Case Strumenti addebito

calcola_addebito(soc:Socio, ini:Data,
fin:Data):Denaro

pre:

```
EXISTS scard, nol |  
    smart_soc(scard, soc) AND  
    nol_smart(nol, scard)
```

post:

questo use-case non modifica il
livello estensionale
result soddisfa la seguente formula:

```
NOLEGGI = { (nol, prezz) |  
    EXISTS scard, in, fn |  
        smart_soc(scard, soc) AND  
        nol_smart(nol, scard) AND  
        prezzo(nol, prezz) AND  
        NoleggioTerminato(nol) AND  
        inizio(nol, in) AND  
        fine(nol, fn) AND  
        ini <= in AND  
        fin >= fn }
```

AND

```
costo_tot = SUM(prezz | EXISTS nol |  
    (nol, prezz) in NOLEGGI)
```

AND

```
rischio_soc = classe_rischio(soc)
```


società_vantaggiose(cat:Categoria):Società[0..*]

// Coming Soon

- Use-Case Strumenti statistiche marketing

cat_preferite(soc:Socio):Categoria[0..*]

// Coming Soon